



Developing Machine Learning models

Gregoire Olivar

Predictive and Descriptive Machine learning models

Contents

1	DEVELOPING MACHINE LEARNING MODELS	5
1.1	Problem description	5
1.2	Machine Learning approach	6
2	Data description	7
2.1	Data bases	7
2.2	Data preparation	8
3	EXPLORATORY DATA ANALYSIS (EDA)	10
3.1	EDA for classification	12
3.1.1	Balance of the dataset	12
3.1.2	Categorical features analysis	13
3.1.3	Numerical features analysis	16
3.1.4	K-Mean Clusterization	18
3.1.5	Data post processing	20
3.1.6	Conclusion of the EDA Classification	21
3.2	EDA for Regression	21
4	Model Evaluation	23
4.1	Model Evaluation for Classification	23
4.2	Model Selection for Classification	24
4.3	Model Evaluation for Regression	26
4.4	Model Selection for Regression	27
4.5	Model Results for Classification	28
4.5.1	Logistic Regression	28
4.5.2	Decision Tree	28
4.5.3	Random Forest	29
4.5.4	KNN	30
4.5.5	Models comparison for classification	30
5	Model results for Regression	32
5.1	Linear Regression	32
5.2	Ridge	33
5.3	Comparison and analysis	33

6 Conclusion

34

List of Figures

1	Different heart disease	5
2	Statistic parameters of some dataset features	8
3	Number of missing values	8
4	Heatmap of missing values	8
5	Feature type before correction	9
6	Feature type after correction	9
7	Explanation of R value	10
8	Visualization of R value	10
9	Correlation matrix	11
10	Correlation matrix with	12
11	Balance of the dataset	13
12	Gender repartition	13
13	Gender distribution through the disease	13
14	Chest pain repartition	14
15	Chest pain distribution through the disease	14
16	Fasting blood sugar repartition	14
17	Fbs distribution through the disease	14
18	RER repartition	14
19	RER distribution through the disease	14
20	EIA Results repartition	15
21	EIA Results distribution through the disease	15
22	Slope repartition	15
23	Slope distribution through the disease	15
24	Thalassemia repartition	15
25	Thalassemia distribution through the disease	15
26	Summary of the Chi-2 Test results	16
27	Numerical features distribution	17
28	Numerical features distribution	18

29	Clusters plot	19
30	Features mean for each clusters	19
31	Outliers number per feature	20
32	Histogram of maximum heart achieved	21
33	Correlation of maximum heart achieved	22
34	Confusion matrix	23
35	Logistic regression	24
36	Decision tree	25
37	Random forest	25
38	Confusion matrix	26
39	4-Folding example	26
40	Linear Regression	27
41	Training and Testing classification report for LR	28
42	Training and Testing classification report for Decision Tree	29
43	Training and Testing classification report for Random Forest	29
44	Training and Testing classification report for Decision Tree	30
45	Comparison table for all the model	30
46	Training and Testing classification report for Decision Tree	31
47	ROC-Curves	32
48	Linear regression simple	33
49	LR cross validation	33
50	Ridge simple	33
51	Ridge cross validation	33
52	Table comparison	33

1 DEVELOPING MACHINE LEARNING MODELS

1.1 Problem description

Heart disease is the leading cause of death in the world, claiming around 18 million lives every year. They are a real threat to public health, causing disorders of the blood vessels, the heart and the risk of heart failure. If they are not diagnosed and treated early, they can have serious and even fatal consequences. It is therefore very important to be able to detect them as early as possible, and certain characteristics in a patient may be indicative of CVD.

There are different kinds of heart disease such as Coronary Heart Disease (CHD) which is the major type of heart condition and occurs to the blockage of arteries because of some fat deposits stopping the blood circulation. Here is an illustration showing various possible heart disease.

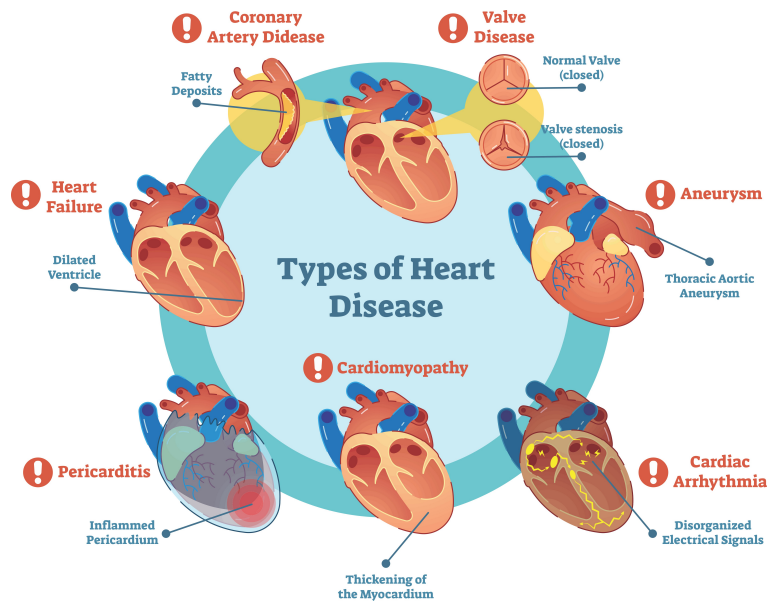


Figure 1: Different heart disease

Today, there are a number of symptoms that may indicate heart disease:

- Chest discomfort or pain, angina (chest pain behind the sternum)
- Palpitations or irregular heartbeat, fatigue or weakness
- Shortness of breath
- Difficulty to walk, Dizziness or lack of balance/coordination
- Nausea or abdominal pain

There are various diagnostic methods for detecting these diseases: An electrocardiogram measures the electrical activity of the heart and may be able to detect irregularities. Moreover, certain blood tests provide levels of cholesterol, glucose and biomarkers such as troponin. These can be revealing depending on their levels. Assessing the heart's performance under physical exertion can also help detect heart disease. Heart disease treatment depends on the type of CVDs and how advanced the disease is. Treatment for heart disease can include:

- Lifestyle changes (eating a diet low in salt and saturated fat, getting more exercise, and not smoking)
- Medicines
- A heart procedure

- Heart surgery

As explained before, the sooner we can detect CVDs, the better it is and the easier the treatment is. The goal of this assignment is to leverage machine learning techniques to improve detection and prevention of CVDs.

1.2 Machine Learning approach

This report will study different machine learning models trying to solve 3 different problems.

Classification problem : Predict whether a patient has heart disease or not

Regression problem: Predict the maximum heart rate achieved using the other features.

Unsupervised Classification Problem: Group the patient into clusters based on similarities in the feature space, without using labels.

. The following steps have been taken to approach the project:

Data Description and Preparation: The first step will be to clean the dataset and ensure its integrity. This step is assessing the quality of the dataset and prepare it to a deeper analysis.

Exploring Data Analysis : In this part we are analyzing the behaviour of each variable and their relation between them and with the target. This step is crucial to identify trends or interactions through the features. Moreover, it is in this part that the last adjustments (based on visualization or statistics tests) are done.

Training, Validation, and Testing Datasets : The dataset will be splitted in different part to train, validate and test the model. First the model is trained by different method and a function will decide which hyperparameters are the best for it. Then, the model is tested and gives various metrics results.

Evaluation : The evaluation will give the results of the models and if they are good to predict what they should predict. It is possible by comparing the result the model gave us and the real result available.

Conclusion In this part we will do the conclusion of our models and what could have been improved. Moreover, we will see the limitations of our projects and why we can have some bad results.

I was a complete beginner in Machine learning and deep learning and used the PRDL and MLLB courses a lot. I also found a lot of useful pieces of information on kaggle that helped me a lot to improve my skills and solve my issues. Copilot AI was also really useful for plotting figures and fixing bugs in my code. I did all my project on my portable laptop on VSCode.

2 Data description

2.1 Data bases

The dataset I will be using is a dataset updated 6 years ago and found on Kaggle. It consists of four different databases: Cleveland, Hungary, Switzerland, and Long Beach V. This dataset was supposed to contain clinical and demographic data on 1025 patients and it can be used to predict the presence of heart disease. We will see that there less patient information than expected. This dataset contains 13 attributes and a target variable:

- **Age** (*numerical*): Age of the patient
- **Sex** (*categorical*): Gender (male : 1 and female : 0)
- **cp** (*categorical*): Type of Chest pain experienced by the patient (0 : typical angina; 1 : atypical angina, 2 : non-anginal pain, 3 : asymptomatic)
- **trestbtps** (*numerical*): Patient's level of blood pressure at resting mode in mm/HG
- **chol** (*numerical*): Serum cholesterol in mg/dl
- **fbs** (*categorical*): Blood sugar levels on fast
- **restecg** (*categorical*): Result of electrocardiogram while at rest in 3 distinct values (0 : Normal, 1 : having ST-T wave abnormality (T wave inversions and/or ST elevation or depression of > 0.05 mV), 2 : showing probable or definite left ventricular hypertrophy by Estes' criteria)
- **thalach** (*numerical*): Maximum heart rate achieved
- **exang** (*categorical*): Angina induced by exercise (0 : NO, 1 : YES)
- **oldpeak** (*numerical*): Exercise-induced ST-depression relative to the state of rest
- **slope** (*categorical*): ST segment measured in terms of slope during peak exercise (0 : up-sloping, 1 : flat, 2 : down-sloping)
- **ca** (*numerical*): The number of major vessels (0-3)
- **thal** (*categorical*): A blood disorder called thalassemia (0 : normal blood flow, 1 : fixed defect (no blood flow in some part of the heart), 2 : reversible defect (a blood flow is observed but it is not normal), 3 : not described)
- **target** (*categorical*): This is what we want to predict (0 : if the patient is normal, 1 : the patient is suffering from heart disease)

The features can be sorted into two types of features : there are the numerical features that are quantitative and represent measurable quantities. For example, the number of major vessels is discrete data but represents countable quantities. The categorical features here are already encoded and represent qualitative information such as gender for example.

Before preparing the data, it is possible to look at some statistics about the dataset quickly:

	age	trestbps	chol	thalach	oldpeak	ca	target
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	131.611707	246.000000	149.114146	1.071512	0.754146	0.513171
std	9.072290	17.516718	51.59251	23.005724	1.175053	1.030798	0.500070
min	29.000000	94.000000	126.000000	71.000000	0.000000	0.000000	0.000000
25%	48.000000	120.000000	211.000000	132.000000	0.000000	0.000000	0.000000
50%	56.000000	130.000000	240.000000	152.000000	0.800000	0.000000	1.000000
75%	61.000000	140.000000	275.000000	166.000000	1.800000	1.000000	1.000000
max	77.000000	200.000000	564.000000	202.000000	6.200000	4.000000	1.000000

Figure 2: Statistic parameters of some dataset features

This table shows statistics about the features and the number of patients studied (1025 patients). For example, we can read the mean age which is 54 years old.

2.2 Data preparation

This step is essential to prepare the data before analyzing it. The dataset was already pretty clean but it was important to check that there were no missing values (if so, there are different ways to handle them : it is possible to replace the missing the value by a known value or we can also delete the whole line for example). It was possible to plot the number of missing values for each feature and a heatmap highlighting in yellow if the line contain missing values and purple if not:

```
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64
```

Figure 3: Number of missing values



Figure 4: Heatmap of missing values

All the heatmap is purple and we can read that there is no missing value.

Another issue that needed to be checked for the dataset is if there are some duplicated rows. Indeed Duplicate entries in a dataset can be problematic for machine learning models as they can introduce bias and impact the model's performance. The result of this check was really surprising : indeed, more than 700 rows were similar. It taught me to never forget this step and be careful on the dataset I could choose for a project. The duplicated lines has been removed in the new dataset.

Finally, the type of features needed to be changed for the categorical features : Indeed, even if they are numerical in term of data type, in term of semantic they are categorical. Thus there type should be changed into *object*, i.e string data type.

```
Resume the data frame
<class 'pandas.core.frame.DataFrame'>
Index: 302 entries, 0 to 878
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0    age        302 non-null    int64
1    sex        302 non-null    int64
2    cp         302 non-null    int64
3    trestbps   302 non-null    int64
4    chol       302 non-null    int64
5    fbs        302 non-null    int64
6    restecg    302 non-null    int64
7    thalach    302 non-null    int64
8    exang      302 non-null    int64
9    oldpeak    302 non-null    float64
10   slope      302 non-null    int64
11   ca         302 non-null    int64
12   thal       302 non-null    int64
13   target     302 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 35.4 KB
```

Figure 5: Feature type before correction

```
<class 'pandas.core.frame.DataFrame'>
Index: 302 entries, 0 to 878
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0    age        302 non-null    int64
1    sex        302 non-null    object
2    cp         302 non-null    object
3    trestbps   302 non-null    int64
4    chol       302 non-null    int64
5    fbs        302 non-null    object
6    restecg    302 non-null    object
7    thalach    302 non-null    int64
8    exang      302 non-null    object
9    oldpeak    302 non-null    float64
10   slope      302 non-null    object
11   ca         302 non-null    int64
12   thal       302 non-null    object
13   target     302 non-null    int64
dtypes: float64(1), int64(6), object(7)
memory usage: 35.4+ KB
```

Figure 6: Feature type after correction

3 EXPLORATORY DATA ANALYSIS (EDA)

EDA is one of the most important stages in data science. Indeed defining and understanding the data enables to draw meaningful conclusions from the data and prepare the data for the machine learning models. EDA provides an overview of the data's structure, distribution and content. In this dataset, the categorical and numerical features can be studied separately. The repartition of the categorical feature can be relevant. Also, it can be interesting to compare each category of a feature with the number of persons with the disease (i.e compare with the target feature). For the numerical features, the linearity and the non-linear relation between them can be visualized. It is useful to look at the histogram of each numerical features and to compare them with if the patient is sick or not.

For both EDAs (Regression and Classification), The first thing when analyzing features is to check if there is some correlation between the numerical variables and the target. A really powerful tool for that is the correlation matrix. This is a table showing the level of correlation between a few variables of a dataset. In other words, it is measuring how "strong" is the linear relation between 2 numerical variables and it is also quantifying itself with a coefficient called Pearson coefficient. This coefficient called R (or r) takes values between -1 and 1. More importantly it gives the intensity and the direction of the correlation. These two graphs illustrate how it works:

r value	Interpretation
$r = 1$	Perfect positive linear correlation
$1 > r \geq 0.8$	Strong positive linear correlation
$0.8 > r \geq 0.4$	Moderate positive linear correlation
$0.4 > r > 0$	Weak positive linear correlation
$r = 0$	No correlation
$0 > r \geq -0.4$	Weak negative linear correlation
$-0.4 > r \geq -0.8$	Moderate negative linear correlation
$-0.8 > r > -1$	Strong negative linear correlation
$r = -1$	Perfect negative linear correlation

Figure 7: Explanation of R value

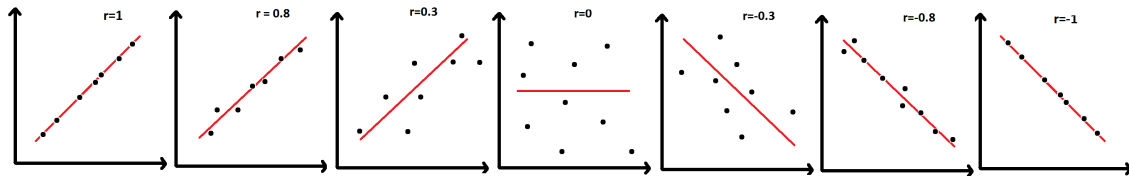


Figure 8: Visualization of R value

We can see that the more the coefficient is close to 1 and -1, the stronger the correlation is. And the sign of the coefficient gives the direction of how one variable increases (r positive) or decreases (r negative) with the other.

The correlation matrix is :

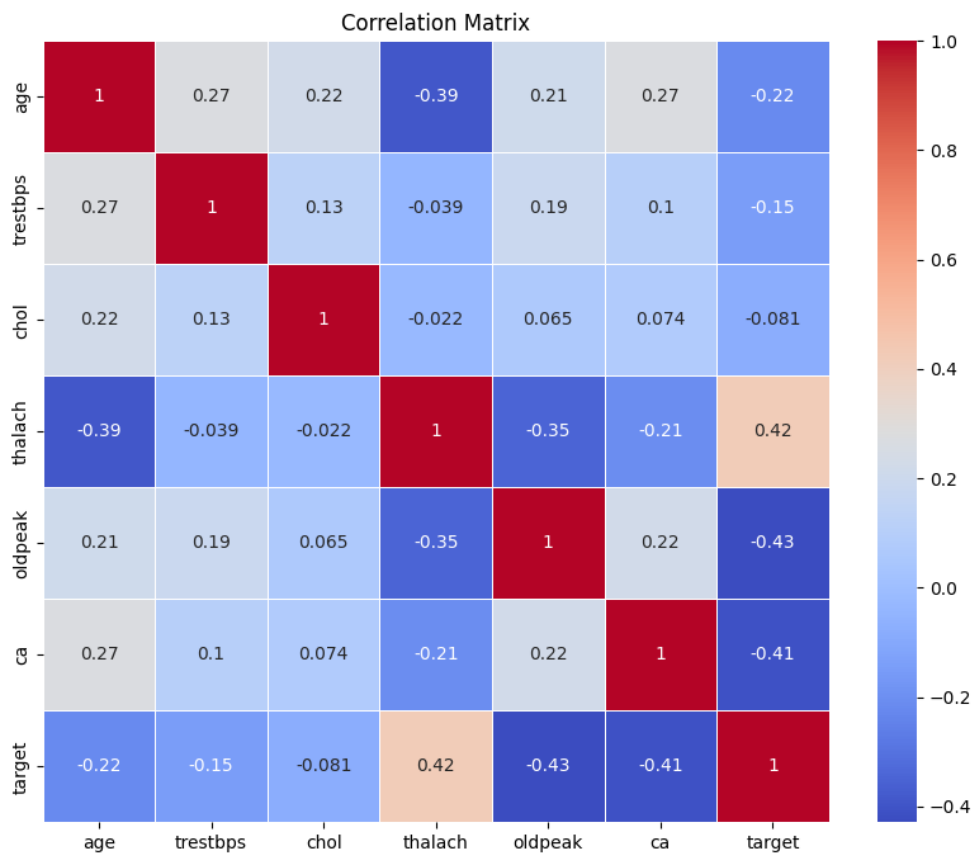


Figure 9: Correlation matrix

First of all, We can directly see that there was no strong or moderate linear correlation between the numerical features. There are a lot of weak correlations. The *thalach* (maximum heart achieved during an effort) and the age are the features with the strongest (but still weak) relationship. Also it has a moderate correlation with the target. Another moderate correlation that the target has, is with oldpeak (Exercise-induced ST-depression relative to the state of rest)

Another tool can be used to visualize linear and non-linear relation is the scatterplot of the numerical features in pair. There is no clear tendency but it was still relevant to plot it.



Figure 10: Correlation matrix with

3.1 EDA for classification

3.1.1 Balance of the dataset

This part will be more specific to the classification problem in the data exploration. It also contains the K-Clusterization giving me some insights on the numerical features.

To start the classification EDA, It is possible to check if the target variable is well balanced.

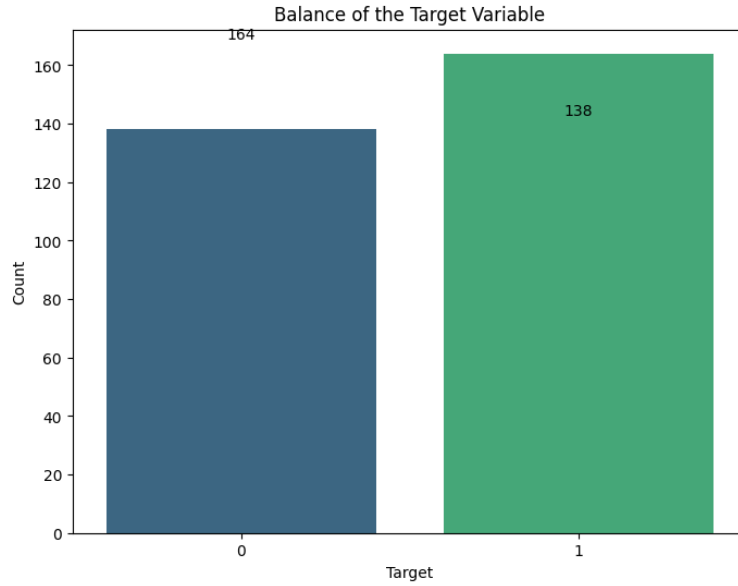


Figure 11: Balance of the dataset

As we can see on this graph, there is almost the same number of patients with and without the disease. It means that our dataset is well-balanced and it will facilitate the training and the evaluation, by avoiding to have a bias : for example if we had 95% of patients without the disease and 5% of patients with the disease, we could easily have an accuracy of 95% just by saying that everybody is not sick. But our model would be completely unable to detect the minor class. Here we do not have this issue.

3.1.2 Categorical features analysis

To analyze the categorical feature, it was interesting to analyze their repartition (this is the chart pie and the histogram) and how is the target feature shared out through the different classes. In other words, we can have the number of patient with and without the disease per class and see how one class is influencing of the target. Let's see them and try to analyze them:

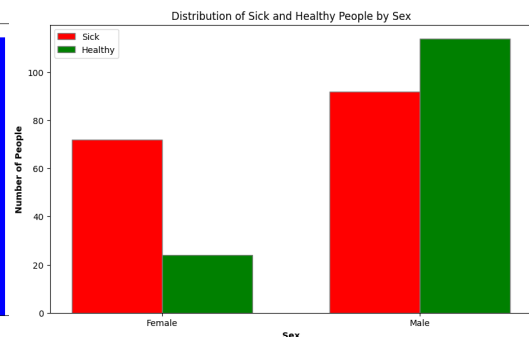
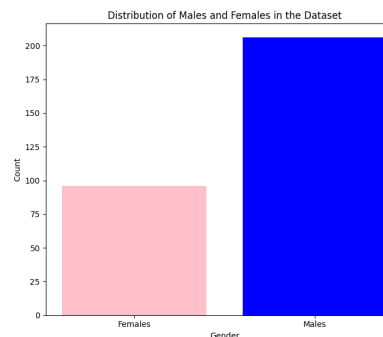
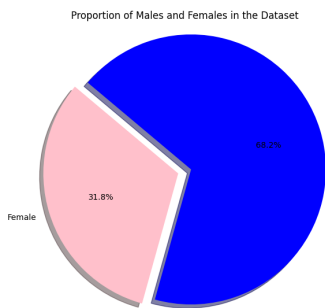


Figure 12: Gender repartition

Figure 13: Gender distribution through the disease

The **Sex** feature shows that there is more men than women in our dataset. Moreover, the second graph shows a significant change in the target repartition depending on the gender of the patient : the females show a higher proportion of heart disease compared to male. Gender seems clearly to be an influential factor in predicting the heart disease.

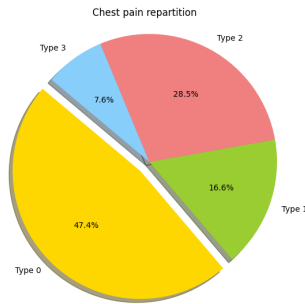


Figure 14: Chest pain repartition

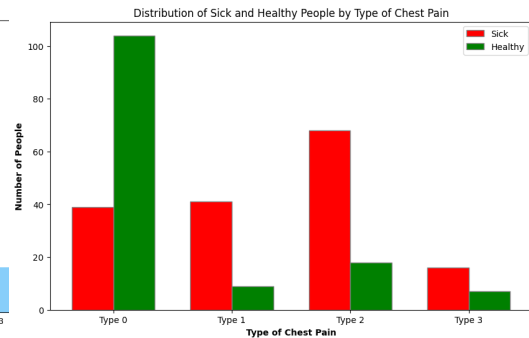
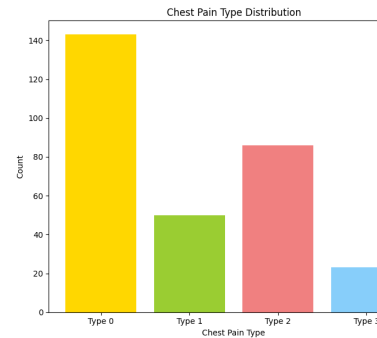


Figure 15: Chest pain distribution through the disease

The **Chest pain repartition** is varied through the 4 chest pain but the type 0 (angina) is representing the patient majority. The repartition of the target is also really relevant : the type 0 has a way smaller proportion of patient with the disease than the other. This clearly suggests that chest pain is an influential factor

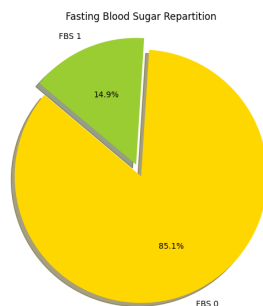


Figure 16: Fasting blood sugar repartition

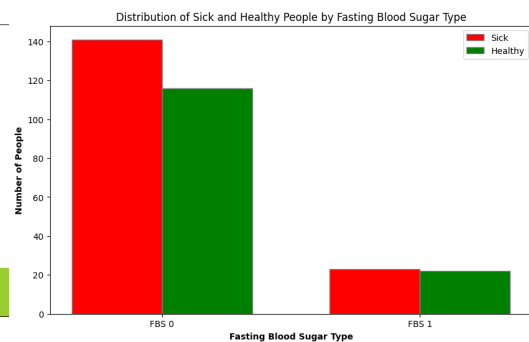
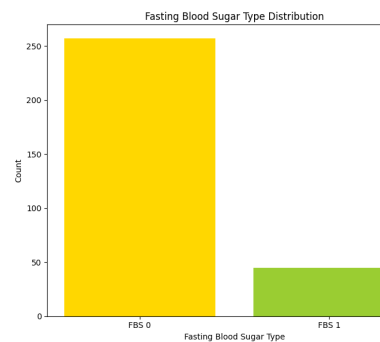


Figure 17: Fbs distribution through the disease

The **Fasting blood sugar** is also not equally shared out: a significant majority of the patients have their fasting blood sugar level below 120 mg/dl. Nevertheless, it does not seem to be an high-influential feature: the repartition with and without the disease seems to be the same for both high and low blood sugar.

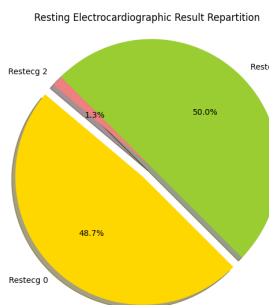


Figure 18: RER repartition

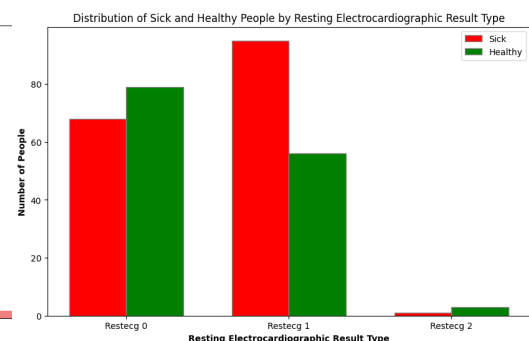
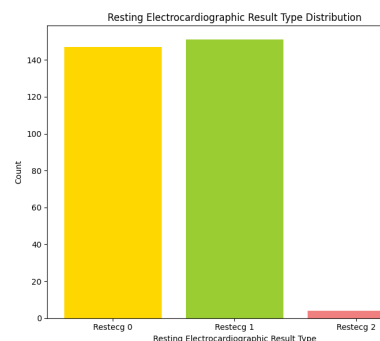


Figure 19: RER distribution through the disease

The **Resting Electrocardiographic Results** shows a huge majority of type 0 and 1. We could have to handle the type 2 rows as there are too few patients in this case to have a significant result on its influence but we are keeping

the rows as ur dataset is pretty small. The disease distribution graph highlights that it is a relevant feature : type 1 displays a higher proportion of heart disease presence.

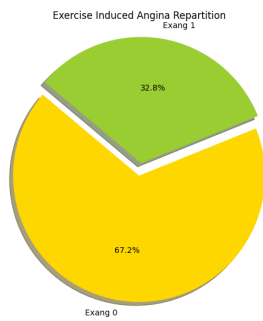


Figure 20: EIA Results repartition

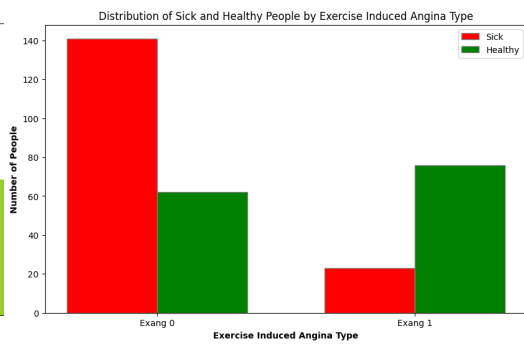
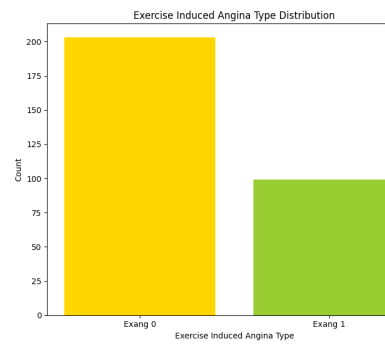


Figure 21: EIA Results distribution through the disease

The **Exercise Induced Angina** is an interesting feature: It has more patients without Angina induced by exercise (type 0) than with (type 1). Nevertheless it seems that there are more patients with the disease that has no Exercise Induced Angina. It might not be a common symptom among the patients in this dataset.

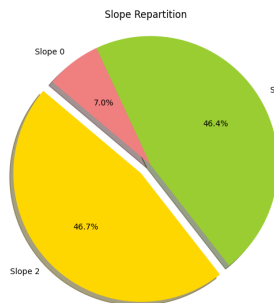


Figure 22: Slope repartition

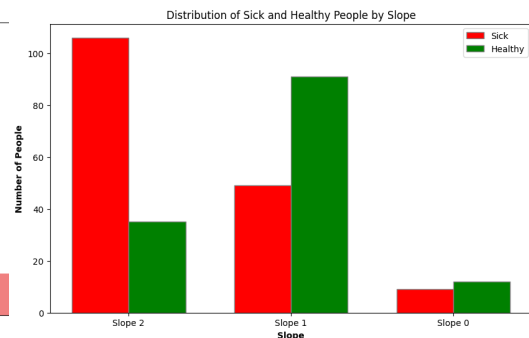
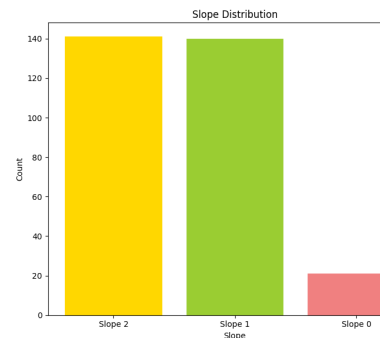


Figure 23: Slope distribution through the disease

The **Slope of the Peak Exercise ST Segment** is important: from the type 0 to the type 1 and 2, the tendance of the disease distribution is completely inverted.

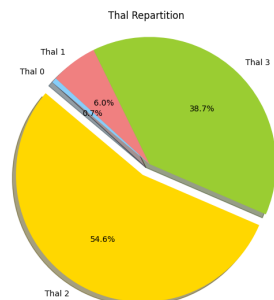


Figure 24: Thalassemia repartition

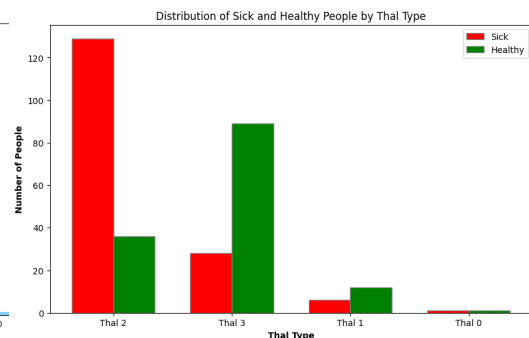
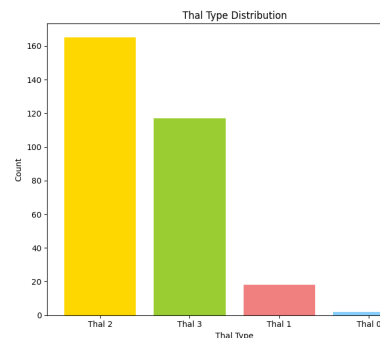


Figure 25: Thalassemia distribution through the disease

The **thalassemia** seems also relevant to keep as the type 2(defect blood flow) is way more sensitive than the type 3 (fixed blood flow) .

Finally, it was useful to submit the categorical features to a final statistic test to decide if it was relevant to keep them or not. The **Chi-2 Test** is a test to identify if a categorical feature is significantly related to the target. It uses two hypotheses such as:

H_0 : Both variables are independent.

H_1 : Both variables are dependent.

A **contingency table** is the starting point of this test. it is a table resuming the occurrences of each combinations between the categorical feature and the target. Then we can use this table to obtain the value expected under the independent hypothesis.

$$\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

Where:

- O_{ij} : Observed values in the contingency table.
- E_{ij} : Expected values under the independence hypothesis, calculated as:

$$E_{ij} = \frac{(\text{Row total of } i) \times (\text{Column total of } j)}{\text{Grand total}}$$

Thus, we have the test statistic χ^2 measuring the gap between the observed values and the expected values. Finally, we can use the p-value to choose if the feature is relevant or not: this is a critical probability able to evaluate if the observed results are compatible under the null hypothesis (both variables are independent). To have a decision boundary, we take an $\alpha = 0.05$ such as :

A **low p-value** ($p < \alpha$): H_0 is rejected in favor of H_1 . This means the observed results are unlikely enough for the null hypothesis to seem implausible. It would mean the feature is relevant as it is dependent with the target. A **high p-value** ($p \geq \alpha$): H_0 is not rejected. This means that the observed results are consistent with H_0 . We won't keep the feature in this case. After implementing this test in Python, this is the code we obtained:

	Variable	Chi2	P-Value	Degrees of Freedom
0	sex	23.083879	1.550855e-06	1
1	cp	80.978762	1.892684e-17	3
2	fbs	0.092408	7.611375e-01	1
3	restecg	9.729682	7.713053e-03	2
4	exang	55.456203	9.556466e-14	1
5	slope	46.889477	6.577783e-11	2
6	thal	84.610318	3.146295e-18	3

Figure 26: Summary of the Chi-2 Test results

This table shows that all my variable seem relevant except one. Indeed, all the features have a p-value under 0.05 except the fast blood sugar (fbs). This is why we won't keep this feature.

3.1.3 Numerical features analysis

The EDA for the classification is also to analyze the numerical features. It can be useful to display histogram in order to have a better comprehension of these features and their distribution with their mean and standard deviation (measure of how dispersed the data is). This is why we plotted the histogram of each numerical feature.

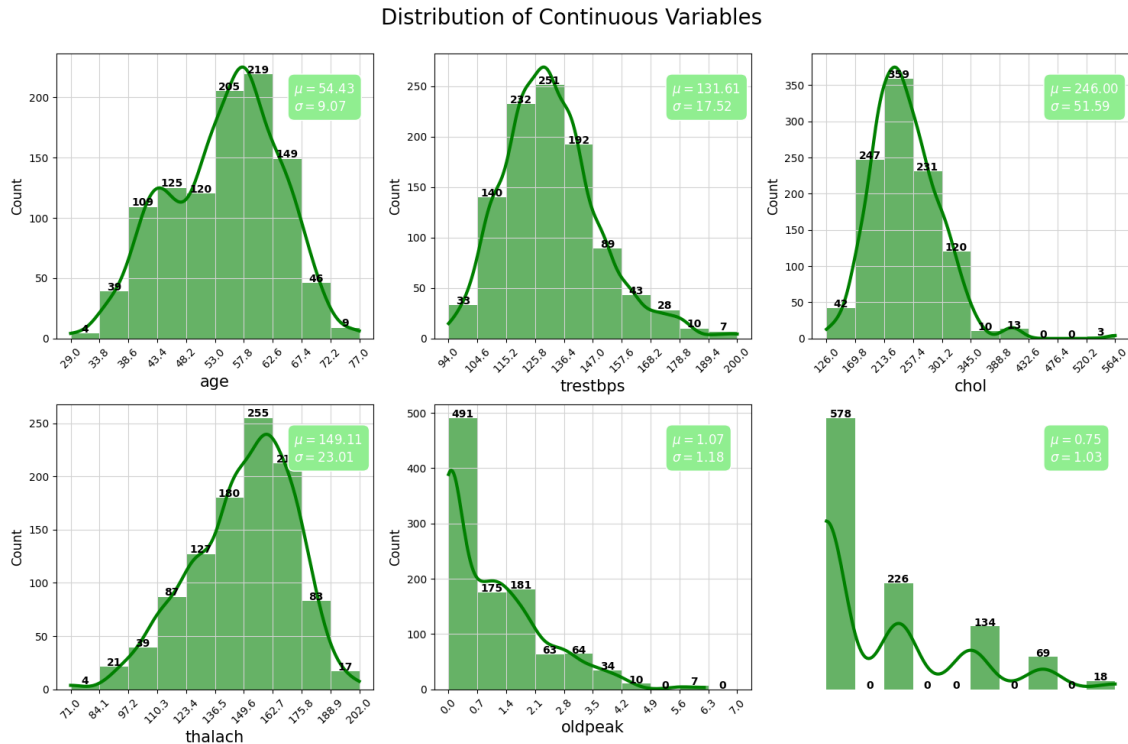


Figure 27: Numerical features distribution

The **Age** distribution is relatively uniform, with a peak around the late 50s. The mean age is 54 years old (SD: 9.08). The **Resting Blood Pressure** has most of its values between 120-140 mm Hg, with a mean of 131.62 mm Hg (SD: 17.54).

The **Serum Cholesterol** levels are mostly in a range between 200 to 300 mg/dl, with a mean of 246.26 mg/dl (SD: 51.83).

The **Maximum Heart Rate Achieved** (thalach): The majority achieve 140-170 bpm during stress tests, with a mean of 149.65 bpm (SD: 22.91).

The **Exercise Induced by Exercise** (oldpeak) has its values mostly close to 0, indicating minimal ST depression for many. The mean is 1.04 (SD: 1.16).

The **Number of major vessels** (ca) is a discrete numerical feature and just show that as the number of major vessel increases, the fewer patients in this case we have.

To compare these datas with the target variable, different graphs can be relevant:

- We can visualize it with **bar plots** to show the average value of each feature grouped by whether the patients have the disease. Thus it can provide a clear representation of how the mean value differs across the 2 classes: if there is a significant difference, it suggest that the feature is a good predictor.

A second useful graph can be the **KDE plots**. As we have seen in Vision Analysis and Deep Learning, this method can plot the probability density function from a histogram or a continuous variable. It estimates the probability density function of each variable with and without the disease. If the distribution are distinct, it suggests that the feature is relevant (because it has a different behavior depending on the target variable).

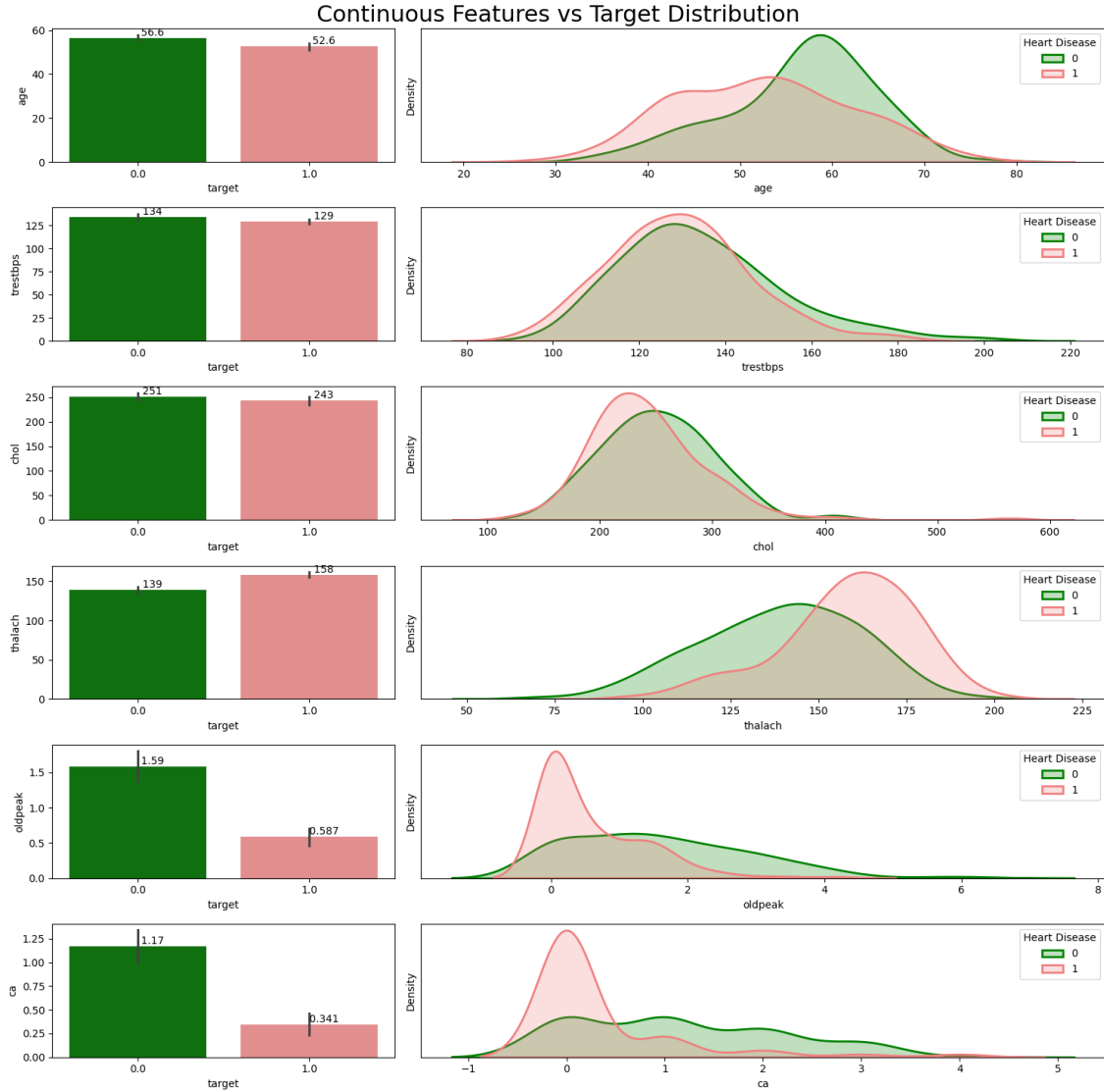


Figure 28: Numerical features distribution

We can see identify in pink and green (respectively with and without the disease) the bar plots with the mean and the density probability function of each feature. The **Maximum heart achieved (thalach)** and the **ST Depression (oldpeak)** show a clear gap between both of their means. Indeed the first one highlights that people with a higher max heart achieved are more sensitive to heart disease. The ST Depression is lower for patient with heart disease. The **age** shows a difference with a higher mean for the people with the disease. The **Serum cholesterol** and the **Resting Blood Pressure (trestbps)** feature was a bit disappointing as their KDEs display overlapping distribution and their box plot were having close means. As our dataset is already really small, We won't delete any numerical features in order to ensure that we will have enough complexity for our model. No columns seem redundant or irrelevant.

3.1.4 K-Mean Clusterization

I used the K-Mean Clusterization as a way to see some correlation in my data. I wanted to see how the model was going to split my patients and how I could relate that to a categorical feature.

This algorithm is an unsupervised learning algorithm used to cluster the data into K groups or clusters based on their similarities. The main goal of this tool is to split the data into groups with similar data and each data from different groups is as different as possible. This technique allows us to find natural group in unlabeled data and can help us to understand better the data. his algorithm can be really useful to discover homogenous clusters and use them as new label. This is how the algorithm works :

- Distribute the K initial points randomly in the space called centroids.
- Each point is associated to the closest distance cluster centroid (this distance can be computed in different ways).
- For each clusters, the centroid are updated as the mean of the cluster points.
- Repeat the two last steps until the centroids do not move anymore.

There are different methods to find the right number of clusters such as the elbow method but with 3 clusters, it is already not working (the clusters are too close to each other and not showing a real separation. I decided to do it only with 2 clusters. After implementing the method on python on my numerical features, I obtained this clusterization:

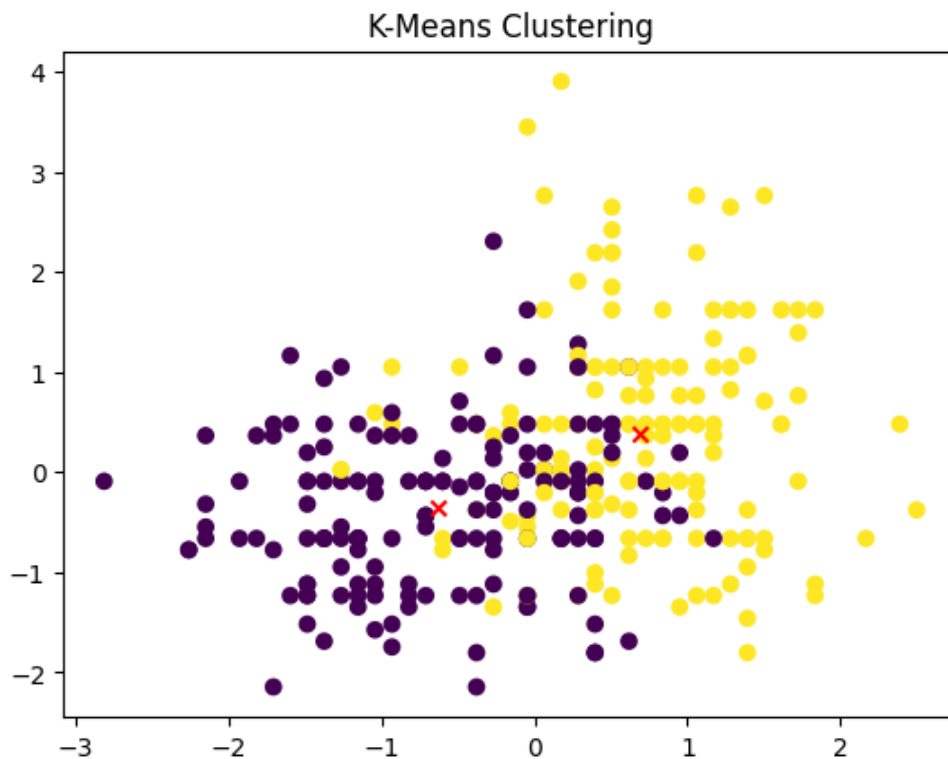


Figure 29: Clusters plot

	age	trestbps	chol	thalach	oldpeak	ca
0	48.765823	125.449367	235.259494	160.867089	0.540506	0.221519
1	60.625000	138.354167	258.833333	137.173611	1.594444	1.263889

Figure 30: Features mean for each clusters

We can see two distinct cloud points. We also obtained the mean for each numerical feature depending on which clusters it belongs. The O cluster has a lower mean age, a lower cholesterol rate, a much lower oldpeak and a higher

max heart rate (thalach) than cluster 1. It seems clearly to correspond to the people with heart disease (if we compare the means to the means of the target vs numerical features plot, they are similar). It means that the numerical features used for this clustering are already informative to distinguish the patient with and without heart disease.

3.1.5 Data post processing

Another thing to identify on the numerical features are the outliers. They are observation values that significantly differ from the other data of the same features. In machine learning, identifying and handling the outliers is important as some models (for example KNN and SVM explained later) can be really sensitive to them, by trying to adapt to these values. A useful method is **Interquartile Range (IQR)**. In this method, we take the difference between the third quartile (value separating the 75% smallest datas) and the first quartile (value separating the 25% smallest datas).

$$\text{IQR} = Q_3 - Q_1$$

It is the range of data between Q1 and Q3, catching the central dispersion of the data. Finally, we define as outliers the values outside of the limit established by the IQR multiplied by a factor 1.5:

Upper limit:

$$Q_3 + 1.5 \times \text{IQR}$$

Lower limit:

$$Q_1 - 1.5 \times \text{IQR}$$

With this method applied to our continuous dataset, we had the number of outliers per features :

```
age      0
trestbps 9
chol     5
thalach  1
oldpeak  5
ca      24
dtype: int64
```

Figure 31: Outliers number per feature

We won't manage the outliers of the major vessel variable (ca) because there are not outliers: as it is a discrete value, the type 3 is fully considered. For the other, there is different way to handle them: We can delete the whole rows. As our dataset is already small, this is not the best idea.

It is also possible to cap the outliers : it involves two steps. First, we need to define threshold based on percentiles (as the 1st and 99th percentile for example). Then we fix the lower outliers to the lower threshold and the upper outliers to the upper threshold.

Another way to reduce the impact of outliers is the transformation with Box-Cox. This method is used to stabilize the variance and make the numerical dataset closer to a normal distribution. It is possible to use it for strictly positive data. The only feature that is not strictly positive is the oldpeak. We add a small constant to each of their values and were able to apply the Box-Cox transformation

$$y' = \begin{cases} \frac{y^\lambda - 1}{\lambda}, & \text{if } \lambda \neq 0, \\ \ln(y), & \text{if } \lambda = 0. \end{cases}$$

y is the original outlier and lambda is a transformation parameter that determines the type of transformation applied. it is found by maximizing the log-likelihood function of the transformed data under the assumption of normal distribution of the feature. This is what I was supposed to do for the outliers, but I finally decided to keep the outliers thinking they won't influence much (it was relevant to change them only for KNN model, but it has good results even with).

The categorical features needed to be **One-Hot encoded**: This method is used to represent categorical data as numerical binary vectors. But we are only encoding the features that have non-inherent ordinal relationship. These features are those with distinct categories (for example "Red", "Blue", "Green") that have no distinct relationship between them. An example of an ordinal relationship would be ("Low", Medium, "High"). Referring to our dataset description, the variable we had to encode is Chest pain (cp), Thallium stress test (thal) and the slope of the peak exercise ST segment (slope).

Finally, we decided to scale our features. The **feature scaling** is the process of normalizing the range of features of the dataset. It ensures equal weightage and that all features contribute to the model. Moreover it can improve the convergence speed for gradient based models. There are different ways to scale our features depending on how we want our data to be scaled. We used Min-Max scaling, that rescaled to the numerical features to a fixed range [0 , 1] using the minimum and maximum of each features.

3.1.6 Conclusion of the EDA Classification

In conclusion of our EDA, we have 303 different patients, 6 useful numerical features and 6 useful categorical features. Some categorical features were more relevant than others and we decided to keep them all except for fast blooding. The numerical features that seem to be the most related to the target are the maximum heart achieved and the ST Depression. The K-Mean Clusterization divided the data into two clusters : it revealing that the clusters correspond more or less to the person with and without the disease. Then we did some final data post-processing to our dataset to scale and encode well our features.

3.2 EDA for Regression

The regression problem is to find the maximum heart rate with the other features. First, it was interesting to have the histogram of this feature :

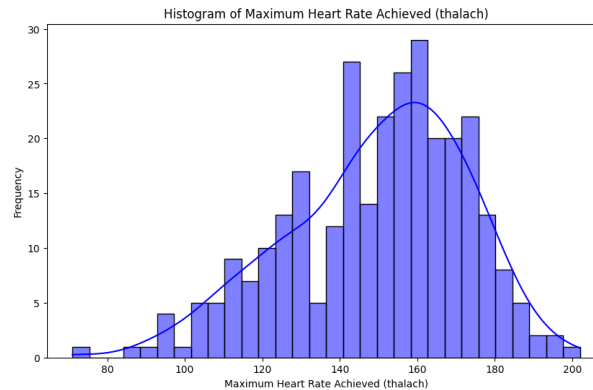


Figure 32: Histogram of maximum heart achieved

The values seem to be well distributed and have a Gaussian-like distribution. There is a low value that we might need to handle (the outlier)

Here is a reminder of the linear correlation of thalach with the other features :

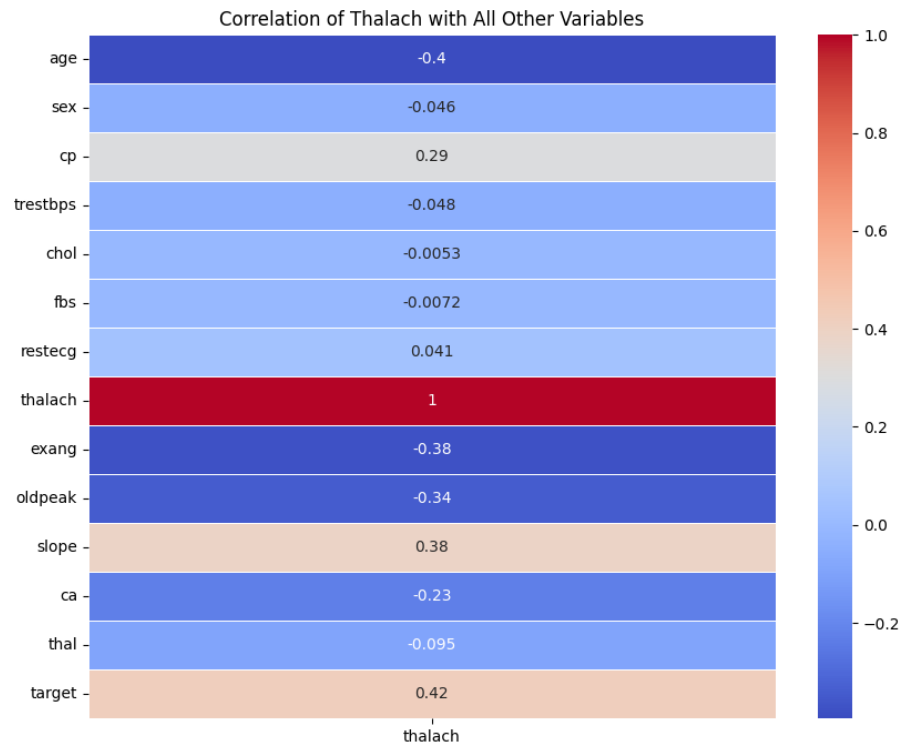


Figure 33: Correlation of maximum heart achieved

We can observe how bad the linear correlation are. I do not expect to have good results from linear regression models. From the classification EDA, I remembered that I had one outlier for this and I decided to delete the line with it as a single outlier can affect the regression line or hyperplane.

I one-encoded the categorical features the same way as for the classification and was able to start my model evaluation.

4 Model Evaluation

4.1 Model Evaluation for Classification

In machine learning, model validation is a useful step to assess how well our model can generalize to unseen data. Without this step, the model might be really good on the set we trained it on but bad on new data. It is called overfitting.

Moreover, each model we have has some parameters called hyperparameters and they can be optimized for its dataset while doing the model validation. There are various methods to find the best hyperparameters and I tried several of them.

My method was to use a method called StratifiedKFold (derived from Kfold) splitting the training data into K splits. StratifiedKFold is doing the same thing as Kfold but it is also ensuring that there are the same proportion of patients with and without the disease in each groups. As our splits are small because of the size of the dataset, it helps a lot to not have bias in the folds. At each iteration, one of the K fold will be used as validation data and the K-1 other one will be used to train the model. And this process is repeated K times, by changing the K fold used for the validation. Its score is the mean of the K scores obtained (I am using different metrics and they will be explained in the testing part). This is called **K-Fold Cross-Validation**. Dividing the dataset into sub-datasets allows the classifier to be evaluated on diverse subgroups and it gives a more robust approach of the model performance.

All the hyper-parameters possibilities are defined in a grid and each combination is evaluated with K-Fold cross-validation described above. Finally, the model and the hyper-parameters combination that obtained the best score are chosen.

The drawback of this method is that the performances of the model are evaluated on a fold from the training data, and this could lead us to choose a model too specific to the training dataset, and that will cause overfitting. I also tried some nested cross validation for my models but the results were poor, probably because of the size of my dataset.

There are different metrics that we will use to evaluate our models.

First of all, we can classify our results into 4 categories: The *True Positive* are the cases where our model correctly predicts that the patient had the disease. The *True Negative* are cases where our model correctly predicts that the patient had not the disease. The *False Positive* are cases where our model predicted incorrectly that the patient had the disease. Finally the *False Negative* cases where our model predicted incorrectly that the patient had not the disease.

A **confusion matrix** summarizes well these statistics in a table. This is an illustration of how it is presented:

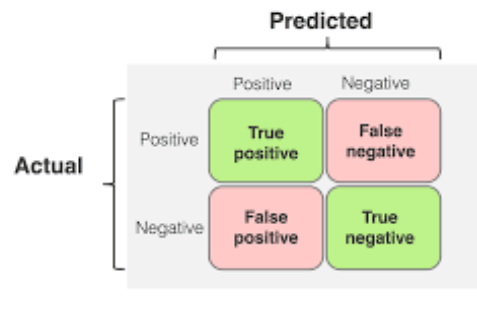


Figure 34: Confusion matrix

In our medical case, it matters that we are predicting the least False negative as possible. Indeed it would be really dangerous to have a sick patient who has been analyzed without heart disease. False positive are also non desirable

but these false alarm are less important than false negative.
Here are different score metrics that can be used:

- The **Accuracy** is measuring the proportion of correct predictions (positives and negatives) out of all the predictions. This metric can be useful as our dataset is balanced.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

The **Precision** is measuring how good is predicting a positive class (accuracy of positive class). In our case, Of all patients where we predicted they had the disease, what fraction really has it?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- The **Recall** is measuring how good is detecting the positive class. In our case, of all patients that has actually heart disease, what fraction did we correctly detect as having it?).

$$\text{Recall} = \frac{TP}{TP + FN}$$

The **F1-Score** is a harmonic mean between the Precision and the Recall. It is useful when we want to balance the Precision and the Recall. Finally, the Recall for the negative class is the metric we want our model to perform the most because it "penalizes" the most the false negative.

4.2 Model Selection for Classification

I had the choice to select different machine-learning model. I used some models based on decision trees and others.

- **Logistic Regression** is not used to solve regression problems as its name could suggest. It is a statistical method that is used for building machine learning models where the dependent variable is dichotomous: i.e. binary (perfect in our case). Logistic regression is used to describe data and the relationship between one dependent variable and one or more independent variables. The logistic function is also known as the sigmoid function and it introduces non-linearity. The value of this function lies between zero and one. If the output of the sigmoid predict a greater value than 0.5 for a patient, the patient can be classified as the positive class, i.e with the heart disease. The figure here shows the decision boundary at 0.5 splitting the classification for the values of the sigmoid :

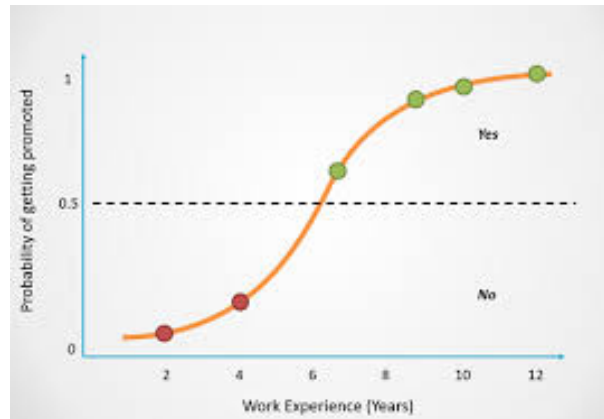


Figure 35: Logistic regression

-**The Decision Tree** is a machine learning algorithm used for classification. It works by dividing the data in several sub-groups based of decision rules made on the features. It can be seen as a tree where the starting poing of the tree represent the totality of the data, and each internal nodes of the tree represent subdivision of the dataset. Finally the last nodes represent the classes selection.

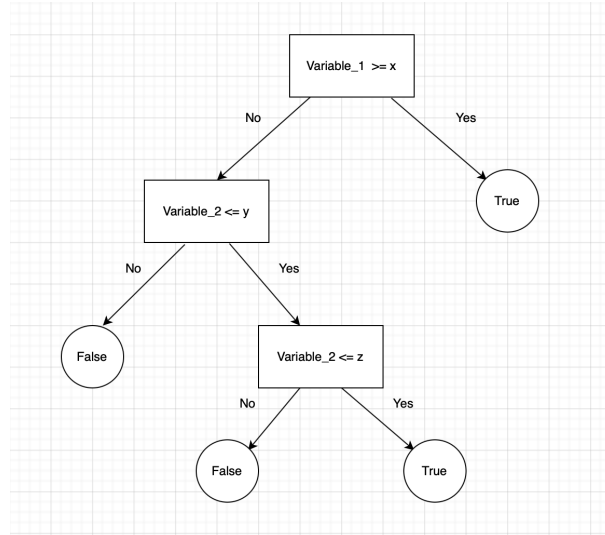


Figure 36: Decision tree

-**The Random Forest** is tree-based machine learning algorithm that leverages the power of multiple decision trees for making decisions (not like Decision tree where there is only one tree). Each individual tree in the random forest spits out a class prediction and the class with the most votes becomes our model's prediction. A large number of relatively uncorrelated models (trees) operating as a committee will outperform any of the individual constituent models.

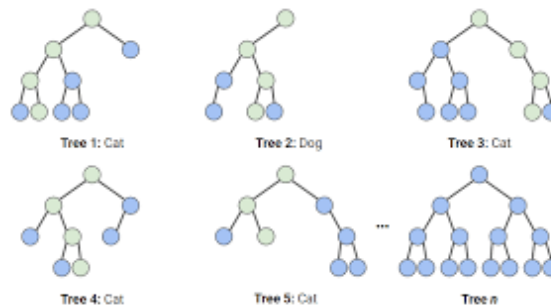


Figure 37: Random forest

KNN Classifier K-Nearest Neighbors is a simple and intuitive supervised learning algorithm, used for classification and regression tasks. This tool keeps in memory the training data and can compute the class when a prediction is needed. We can see the training dataset as a n-dimension space where each sample is a point. As soon as a new prediction is needed, the algorithm computes the distance between this point and the other points and identify the K nearest point. There is different ways to compute this distance : it can be a manhattan distance or an euclidean distance for example. My hyperparameters function will be able to choose which one are the best. The majority class of its K nearest neighbours is attributed to this point. This algorithm depends on the K chosen, as a small K will make the algorithm really sensitive to the noise and a big K can erase the local tendencies.

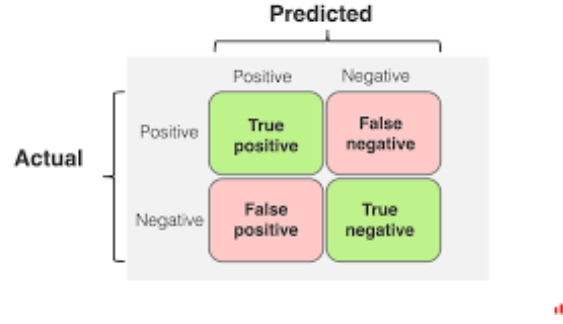


Figure 38: Confusion matrix

4.3 Model Evaluation for Regression

For this part, I decided not to use the same function for the hyperparameters of my model. I did some cross-validation on the whole dataset. Indeed, I used the classical evaluation method where the whole dataset is splitted into 5 folds and at each iterations, 4 of the fold are used to train the model and one is uses to test it. Five is often used as it allow the classic 80% train and 20% test distribution for each iteration. After doing this for each different fold, we have 5 scores and we can do the mean of it. Here is a small illustration about it :

4-fold validation (k=4)



Figure 39: 4-Folding example

I selected this method because the function I explained previously was causing a lot of overfitting. I also tried to do the models with a classic single data division (80% for training and 20% for testing) but the results were less correct obviously : Cross-validation is more robust and decrease the bias that could have been introduced in a unique data separation.

There are different metrics to evaluate a regression model :

Mean Squared Error (MSE): MSE measures how close a regression line is to a set of points by squaring the residuals. It is always larger than MAE and cannot be directly compared to it. However, MSE values can be used to compare different regression models to determine the best fit for the data. Unlike MAE, where errors contribute linearly, MSE grows quadratically with large errors. This makes MSE more sensitive to outliers, heavily penalizing predictions that differ significantly from true values. A higher MSE indicates a model struggling with large prediction errors.

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Mean Absolute Error (MAE): MAE measures the average absolute difference between true and predicted values. It treats all errors equally, without squaring residuals, making it more robust to outliers than MSE. A small MAE indicates excellent predictions, while a large MAE suggests poor performance. However, this robustness applies only when large errors are no more critical than smaller ones.

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

The Root Mean Squared Scaled Error (RMMSE) RMSSE measures prediction accuracy by scaling the root mean squared error (RMSE) relative to the scale of the data. Unlike RMSE, which is affected by the units of measurement, RMSSE is unitless and can be compared with the target (i.e the maximum heart rate).

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}$$

4.4 Model Selection for Regression

It is important to test different models to estimate the maximum heart rate. I tried two different models : **The linear regression model** is a statistic model trying to establish a linear relation between the input features and the target features. It can be written like this :

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \epsilon$$

The beta coefficients are the weights of each associated input feature. And epsilon is the interception. The model learns the optimal weight values by minimizing the least squared error (the cost function) at each iteration. This minimization is done using an algorithm called the gradient descent : it adjusts the weights following the cost function gradient in the inverted direction. This direction decreases the cost function. Here is an illustration of linear regression :

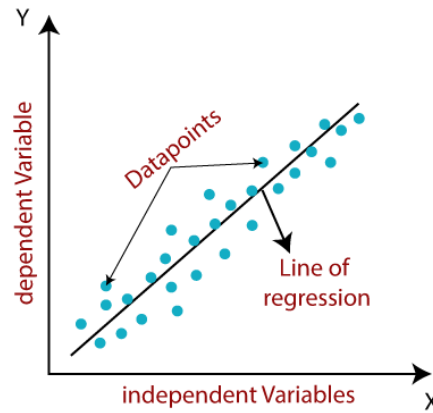


Figure 40: Linear Regression

The Ridge model also known as L2 Regularized Regression is basically a linear regression with regularization term in its cost function. Indeed it modifies the cost function by adding a penalty term proportional to the squared of the magnitude of the coefficient. The cost function of a linear regression was :

$$J(\beta) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 = \frac{1}{2m} \sum_{i=1}^m (y_i - (\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}))^2$$

And with the regularization term, it becomes:

$$J(\beta) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \frac{\lambda}{2} \sum_{j=1}^p \beta_j^2$$

This penalty term is a hyperparameter and discourage large coefficients that can cause overfitting. This parameter needs to be optimal if we want the model to perform the best : if it is too small, it won't penalize much the linear regression weights and if it is too big, coefficient can shrink towards 0 and the model can underfit.

4.5 Model Results for Classification

In this part we will see the results of all our model (confusion matrix, classification report containing recall, f1-score, other metrics and some curves that can help us to understand the results). A simple way to see if our model was overfitting is to compare the average training scores and the average testing scores : Indeed, if the training scores are significantly higher than the testing score, it means that the model is too specific to its training data and is overfitting.

4.5.1 Logistic Regression

The function I did to optimize the Logistic Regression was not working well and it was underfitting. I decided to decrease λ the regularization parameter. It is a parameter that penalizes the weight of some features in the gradient descent algorithm. It was probably decreasing too much the weight of the variable and the model was not able to learn anymore. With a smaller lambda, I obtained these pretty decent results :

REPORT CLASSIFICATION LOGISTIC REGRESSION ON THE TEST DATASET				
	precision	recall	f1-score	support
0	0.81	0.79	0.80	28
1	0.82	0.85	0.84	33
accuracy			0.82	61
macro avg	0.82	0.82	0.82	61
weighted avg	0.82	0.82	0.82	61
REPORT CLASSIFICATION LOGISTIC REGRESSION ON THE TRAINED DATASET				
	precision	recall	f1-score	support
0	0.85	0.70	0.77	110
1	0.78	0.89	0.83	131
accuracy			0.80	241
macro avg	0.81	0.80	0.80	241
weighted avg	0.81	0.80	0.80	241

Figure 41: Training and Testing classification report for LR

The weighted average training and testing scores are really close to each other and are good : the model doesn't appear to be overfitting.

The recall for the class 1 is 85%, meaning that 85% of the patients with the disease have been detected correctly.

4.5.2 Decision Tree

This model also had an issue with the cross-validation function as our model was also overfitting. In Decision Tree, the deeper the tree is, the more complex the model is and the model is more likely to overfit. This is a hyperparameter

that we can change and my function was suppose to find the best max depth automatically. I suppose that the size of my dataset is also not helping the function to give the best results. By changing the maximum depth from 3 to 2, the model presented consistent and correct results:

REPORT CLASSIFICATION DECISION TREE ON THE TEST DATASET					
	precision	recall	f1-score	support	
0	0.70	0.82	0.75	28	
1	0.82	0.70	0.75	33	
accuracy			0.75	61	
macro avg	0.76	0.76	0.75	61	
weighted avg	0.76	0.75	0.75	61	
REPORT CLASSIFICATION DECISION TREE ON THE TRAINED DATASET					
	precision	recall	f1-score	support	
0	0.76	0.72	0.74	110	
1	0.77	0.81	0.79	131	
accuracy			0.77	241	
macro avg	0.77	0.76	0.76	241	
weighted avg	0.77	0.77	0.77	241	

Figure 42: Training and Testing classification report for Decision Tree

This model is also not overfitting. The recall for positive class is not as good as the logistic regression: it is at 70%.

4.5.3 Random Forest

The random forest gave good results and was not overfitting with the hyperparameters given by my function.

REPORT CLASSIFICATION RANDOM FOREST ON THE TEST DATASET					
	precision	recall	f1-score	support	
0	0.85	0.82	0.84	28	
1	0.85	0.88	0.87	33	
accuracy			0.85	61	
macro avg	0.85	0.85	0.85	61	
weighted avg	0.85	0.85	0.85	61	
REPORT CLASSIFICATION RANDOM FOREST ON THE TRAINED DATASET					
	precision	recall	f1-score	support	
0	0.89	0.80	0.84	110	
1	0.85	0.92	0.88	131	
accuracy			0.86	241	
macro avg	0.87	0.86	0.86	241	
weighted avg	0.87	0.86	0.86	241	

Figure 43: Training and Testing classification report for Random Forest

Obviously the results of random forest are better than one decision tree. We have a high recall for the positive class (88%) and it presents good scores overall.

4.5.4 KNN

The KNN did not overfit with the best hyperparameters given by my function. The recall of the positive class is not

REPORT CLASSIFICATION KNN ON THE TEST DATASET					
	precision	recall	f1-score	support	
0	0.76	0.93	0.84	28	
1	0.93	0.76	0.83	33	
accuracy			0.84	61	
macro avg	0.85	0.84	0.84	61	
weighted avg	0.85	0.84	0.84	61	
REPORT CLASSIFICATION KNN ON THE TRAINED DATASET					
	precision	recall	f1-score	support	
0	0.83	0.77	0.80	110	
1	0.82	0.86	0.84	131	
accuracy			0.82	241	
macro avg	0.82	0.82	0.82	241	
weighted avg	0.82	0.82	0.82	241	

Figure 44: Training and Testing classification report for Decision Tree

that good (76%) compared to the recall of the negative class (93%), representing the fraction of person that really have not the disease through all the person classified without the disease.

4.5.5 Models comparison for classification

First of all, let's resume in a table the average testing result of the four models.

	Model	Precision (%)	Recall (%)	F1 Score (%)	Recall Positive Class (%)
0	Logistic Regression	82.0	82.0	81.9	84.8
1	Decision Tree	76.4	75.4	75.4	69.7
2	Random Forest	85.2	85.2	85.2	87.9
3	KNN	85.2	83.6	83.6	75.8

Figure 45: Comparison table for all the model

As we can see the random forest presents the best overall results and also the best recall for the positive class. This was my most important scoring as it ensures that there are as few false negatives as possible. This model is be efficient on small datasets by meaning the predictions and decreasing the deviation while avoiding overfitting. The second and third best models are KNN and the logistic regression. Their recall are above 80%. I was pretty surprised of the the logistic regression as it is parametric method and did not expect it to work that well. The decision tree is the simplest non-parametric model and present the worst precision and recall probably because of its simplicity.

Now let's see the confusion matrix :

This matrix shows well the answer of our problem: the cases we want to avoid the most are the bottom left cases. It is the patients diagnosed without heart disease that actually have heart disease. The best model for that is the Logistic regression, but as my dataset is really small and the test dataset is even smaller, we can't guarantee that is will be the

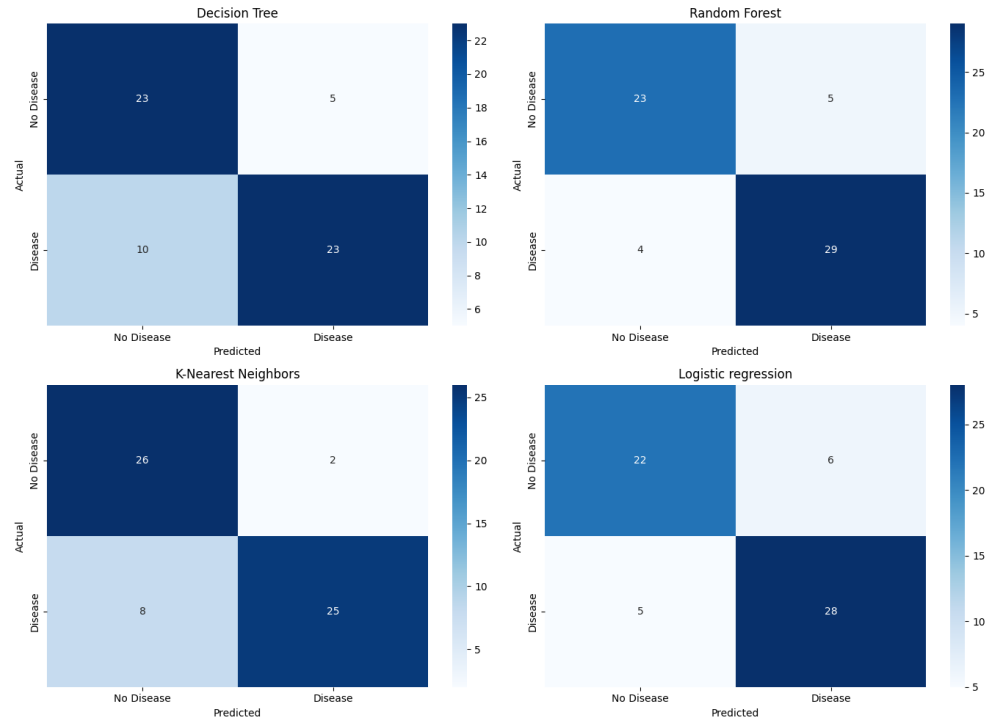


Figure 46: Training and Testing classification report for Decision Tree

best model for this metric. Random Forest also predicts few false negatives.

The last thing is the ROC-Curve :

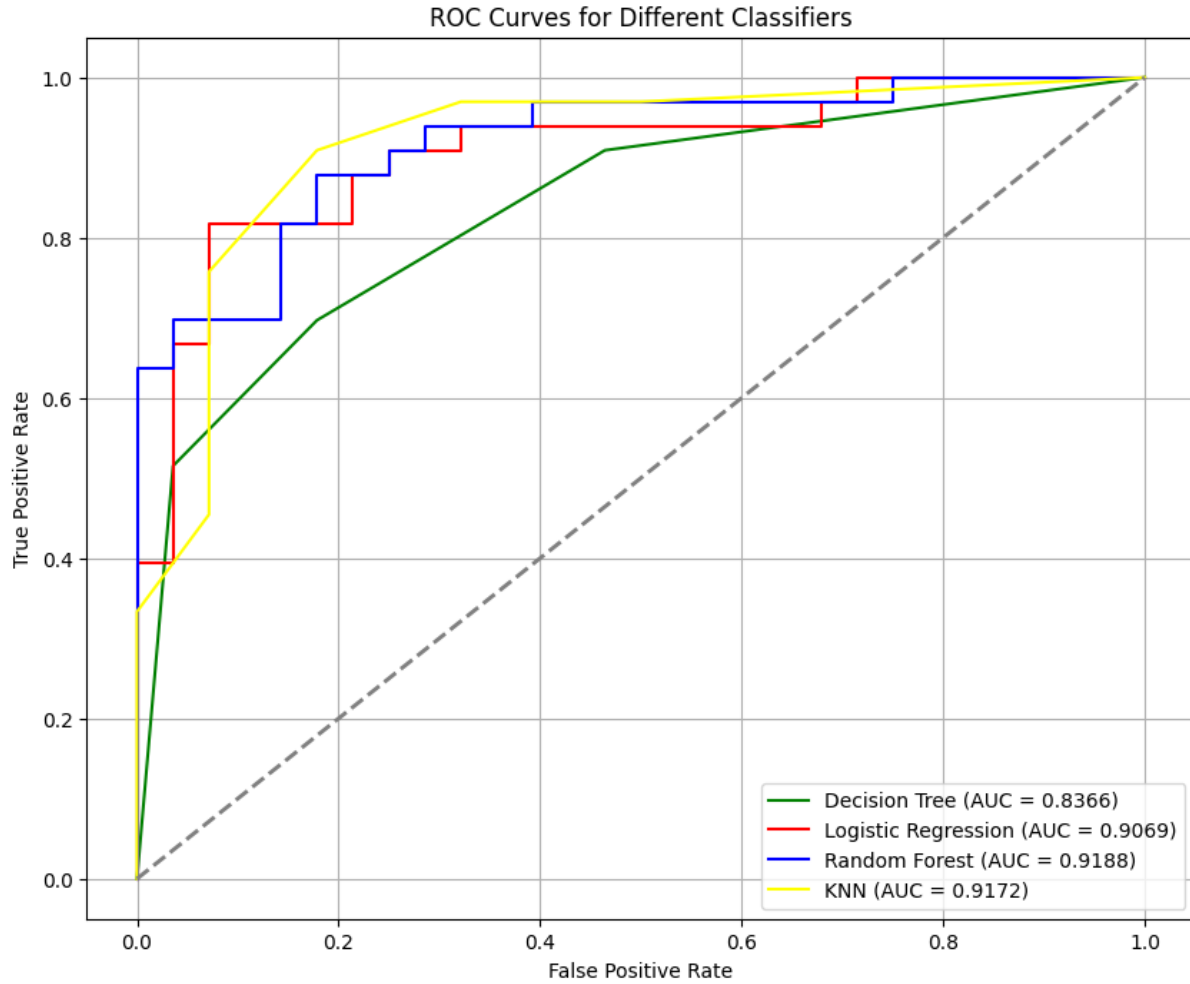


Figure 47: ROC-Curves

The decision tree presents the results as expected with the smallest area under the curve for decision tree. The three other models present decent results.

5 Model results for Regression

5.1 Linear Regression

I did different approach for this model. I first tried to do a simple linear regression following the single data division 80/20.


```
Mean Squared Error (MSE): 318.4769238004894
Mean Absolute Error (MAE): 13.950115291997106
Root Mean Squared Error (RMSE): 17.845921769426464
```

Figure 48: Linear regression simple

```
Cross-Validation Mean Squared Error (MSE): 378.6469
Cross-Validation Mean Absolute Error (MAE): 15.1805
Cross-Validation Root Mean Squared Error (RMSE): 19.4589
```

Figure 49: LR cross validation

As we can see, the cross-validation is not giving better results than the simple linear regression. We have an RMSE around 18 beat per minutes. It is a big error for my maximum heart rate feature (if we look at the distribution of the feature it goes from 80 Bpm to close to 200 Bpm) and shows that the linear regression is not working well.

5.2 Ridge

I thought that the ridge model will be working well or at least better than the linear regression

```
Mean Squared Error (MSE): 317.8848596796775
Mean Absolute Error (MAE): 13.92925484014991
Root Mean Squared Error (RMSE): 17.82932583357199
```

Figure 50: Ridge simple

```
Best Hyperparameters for Ridge Regression: {'alpha': 25}
Mean Squared Error (MSE): 319.6260479024638
Mean Absolute Error (MAE): 13.686376819443824
Root Mean Squared Error (RMSE): 17.87808848569846
```

Figure 51: Ridge cross validation

The Ridge regression gave the same results, with a RMMSE of around 18 BPM. It is finally normal as it seems that our model underfit : the regularization term is useless.

5.3 Comparison and analysis

	Model	MSE	MAE	RMSE
0	Linear Regression	318.476924	13.950115	17.845922
1	Linear Regression with cv	378.646943	15.180501	15.180501
2	Ridge Regression	317.884860	13.929255	17.829326
3	Ridge with cv	318.457211	13.929255	17.878088

Figure 52: Table comparison

The four models present poor results and the regression is not useful for anyone, it was just to try to implement some regression models. I could have tried to understand more why the results were deceiving but I decided to use my time to improve my classification. Moreover the dataset is really small : these kind of dataset often fail to capture the diversity and variability of real-world data.

6 Conclusion

I was a complete beginner in Machine learning and this project helped me a lot to understand various concepts of this field. I fully understood the importance of the data pre and post-processing, and that exploring the data could give us some really interesting insights. For the classification task, I had several model able to have a score above 80%, that is a decent score. I could have tried other models, such as gradient boosting to see if it was possible to improve my results. The limitation of this project is the size of the dataset. Indeed, with only 300 patients, it could easily cause overfitting, and all our results need to be taken with a grain of salt. The regression was not really relevant but it was also interesting to see different methods to implement these models and the croos-validation. Future work could include gathering more data, exploring advanced models such as ensemble learning or neural networks, and further refining the data preprocessing steps to handle outliers and imbalanced features in a more efficient way.